

RENSSELAER MECHATRONICS

Communication: Sending Data

Overview:

“Monitor and Tune” mode in Simulink allows data to be sent and received from the target board (historically called external mode). This communication protocol has overhead which limits how fast data can be transferred. For example, an Arduino Mega in external mode the maximum sampling rate is around 30Hz with 2015a or earlier and up to 1KHz with later versions of MATLAB. To obtain information at a faster rate the data can be send directly from the target without using “Monitor and Tune”.

This lab explores sending data with Simulink using both built in blocks and with the Arduino IDE. It will also review different data types.

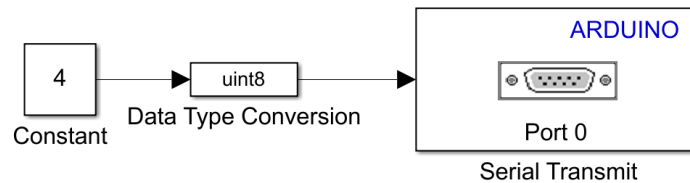
Part 1: Sending 8-bit data with Simulink

Objectives:

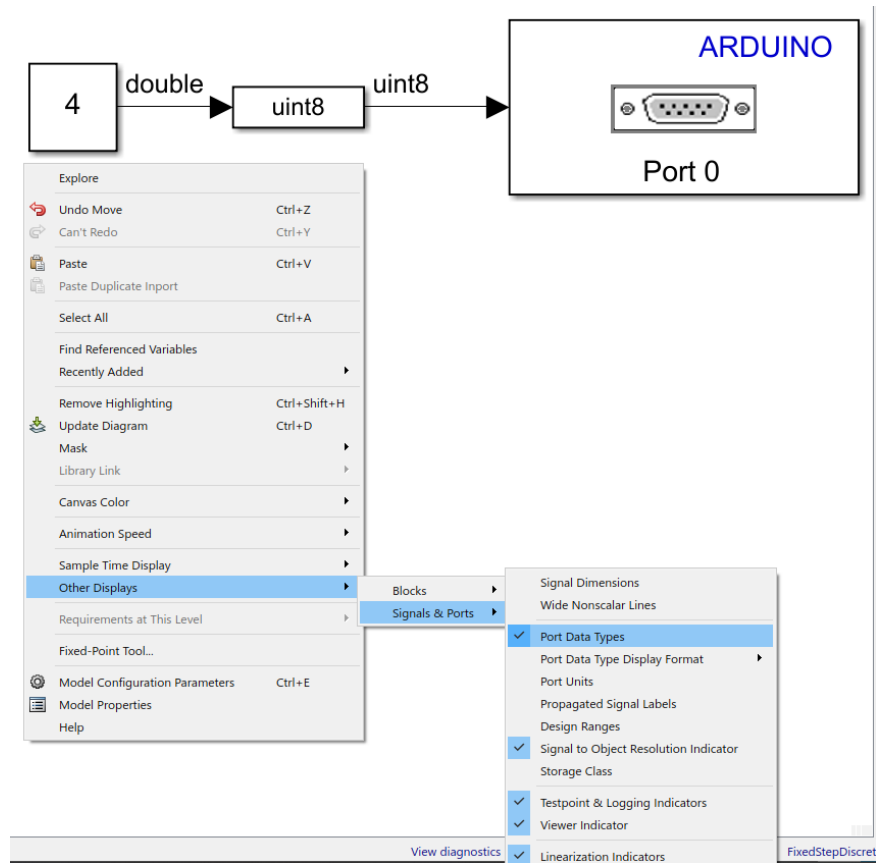
- Send 8-bit data over the serial link using the supported Simulink blocks

Simulink Model:

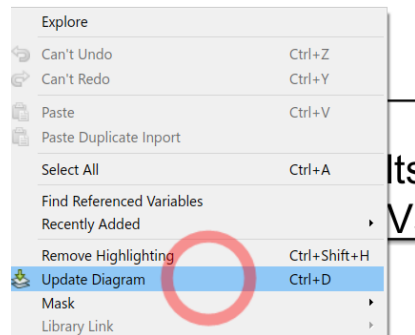
- Build the Simulink diagram from a “Constant”, “Data Type Conversion” and a “Serial Transmit” block. Be sure to use the Demo Simulink diagram and create this new one from that one.



- The Simulink Arduino library block “Serial Transmit” can be used to send single bytes at a time. To see this right-click in the main windows and select “Other Displays – Signals & Ports – Port Data Types”:

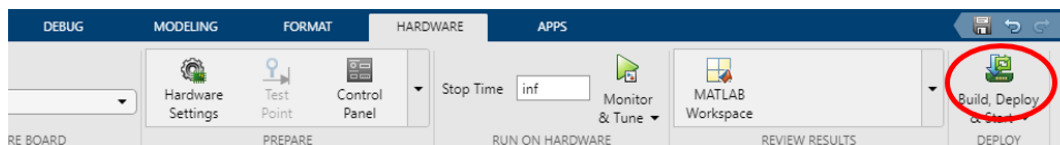


- Press “Control+D” to update the diagram see the data types, right-click in the diagram to Update the Diagram:

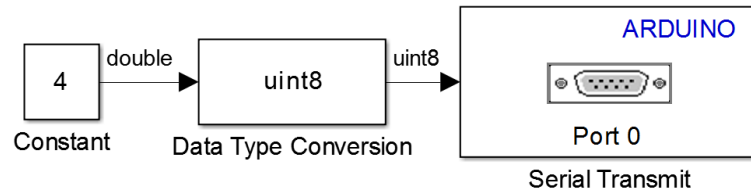


The input data type for a Serial Transmit is uint8 (a single byte, 8 bits of data). This means that data must be converted to uint8 before sending across the serial line with this block.

- Build and run the following Simulink diagram.
 - Click the 'Build, Deploy & Start' button:



- The baud rates for the ports are set in the Configuration Parameters in the Run On Target Hardware tab – by default they are 9600
- Set the sample time of the simulation to .01 second – this means it will send the number 4 in binary every 0.01 seconds through Port 0 (which is the USB cable)



Reading the Data with MATLAB:

Create an m-file with the following code. Use the COM port of your device. This code will open the com port and store the results in the variable d1. You will need to modify the code below based on the COM port of your device. (the end of this lab has a section [How do I find my COM port?](#))

```

%% Read from Serial port:
clc, clear all % clear screen, clear all variables
s = serialport('COM45',9600); % Specify your com port
% Open Serial Port:
fopen(s);
% Read 30 data points
d1=(fread(s, 10, 'uint8'))' % display data

clear s % closes com port
  
```

The output on the MATLAB command line should be:

```

Command Window

d1 =

    4    4    4    4    4    4    4    4    4    4
  
```

To display the ASCII equivalent characters, or the binary form of ones and zeros:

```

%% Read from Serial port:
clc, clear all % clear screen, clear all variables
s = serialport('COM45',9600); % Specify your com port
% Open Serial Port:
fopen(s);
% Read 30 data points
d1=(fread(s, 10, 'uint8'))' % display data
d1_characters=char(d1)      % display data as characters
d1_binary=dec2bin(d1)      % display ones and zeros

clear s % closes com port
  
```

The result

```
d1 =

    4     4     4     4     4     4     4     4     4     4

d1_characters =

'EEEEEEEEEE'
```

Notice 'd1_characters' will display the characters that the bytes represent instead of the numerical binary values. The characters represented by a byte can be found from a Ascii character table:

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	##32;	Space	64	40	100	##64;	@	96	60	140	##96;	`
1	1	001	SOH (start of heading)	33	21	041	##33;	!	65	41	101	##65;	A	97	61	141	##97;	a
2	2	002	STX (start of text)	34	22	042	##34;	"	66	42	102	##66;	B	98	62	142	##98;	b
3	3	003	ETX (end of text)	35	23	043	##35;	#	67	43	103	##67;	C	99	63	143	##99;	c
4	4	004	EOT (end of transmission)	36	24	044	##36;	\$	68	44	104	##68;	D	100	64	144	##100;	d
5	5	005	ENO (enquiry)	37	25	045	##37;	%	69	45	105	##69;	E	101	65	145	##101;	e
6	6	006	ACK (acknowledge)	38	26	046	##38;	&	70	46	106	##70;	F	102	66	146	##102;	f
7	7	007	BEL (bell)	39	27	047	##39;	'	71	47	107	##71;	G	103	67	147	##103;	g
8	8	010	BS (backspace)	40	28	050	##40;	(	72	48	110	##72;	H	104	68	150	##104;	h
9	9	011	TAB (horizontal tab)	41	29	051	##41;	)	73	49	111	##73;	I	105	69	151	##105;	i
10	A	012	LF (NL line feed, new line)	42	2A	052	##42;	*	74	4A	112	##74;	J	106	6A	152	##106;	j
11	B	013	VT (vertical tab)	43	2B	053	##43;	+	75	4B	113	##75;	K	107	6B	153	##107;	k
12	C	014	FF (NP form feed, new page)	44	2C	054	##44;	,	76	4C	114	##76;	L	108	6C	154	##108;	l
13	D	015	CR (carriage return)	45	2D	055	##45;	-	77	4D	115	##77;	M	109	6D	155	##109;	m
14	E	016	SO (shift out)	46	2E	056	##46;	.	78	4E	116	##78;	N	110	6E	156	##110;	n
15	F	017	SI (shift in)	47	2F	057	##47;	/	79	4F	117	##79;	O	111	6F	157	##111;	o
16	10	020	DLE (data link escape)	48	30	060	##48;	0	80	50	120	##80;	P	112	70	160	##112;	p
17	11	021	DC1 (device control 1)	49	31	061	##49;	1	81	51	121	##81;	Q	113	71	161	##113;	q
18	12	022	DC2 (device control 2)	50	32	062	##50;	2	82	52	122	##82;	R	114	72	162	##114;	r
19	13	023	DC3 (device control 3)	51	33	063	##51;	3	83	53	123	##83;	S	115	73	163	##115;	s
20	14	024	DC4 (device control 4)	52	34	064	##52;	4	84	54	124	##84;	T	116	74	164	##116;	t
21	15	025	NAK (negative acknowledge)	53	35	065	##53;	5	85	55	125	##85;	U	117	75	165	##117;	u
22	16	026	SYN (synchronous idle)	54	36	066	##54;	6	86	56	126	##86;	V	118	76	166	##118;	v
23	17	027	ETB (end of trans. block)	55	37	067	##55;	7	87	57	127	##87;	W	119	77	167	##119;	w
24	18	030	CAN (cancel)	56	38	070	##56;	8	88	58	130	##88;	X	120	78	170	##120;	x
25	19	031	EM (end of medium)	57	39	071	##57;	9	89	59	131	##89;	Y	121	79	171	##121;	y
26	1A	032	SUB (substitute)	58	3A	072	##58;	:	90	5A	132	##90;	Z	122	7A	172	##122;	z
27	1B	033	ESC (escape)	59	3B	073	##59;	;	91	5B	133	##91;	[	123	7B	173	##123;	{
28	1C	034	FS (file separator)	60	3C	074	##60;	<	92	5C	134	##92;	\	124	7C	174	##124;	
29	1D	035	GS (group separator)	61	3D	075	##61;	=	93	5D	135	##93;	]	125	7D	175	##125;	}
30	1E	036	RS (record separator)	62	3E	076	##62;	>	94	5E	136	##94;	^	126	7E	176	##126;	~
31	1F	037	US (unit separator)	63	3F	077	##63;	?	95	5F	137	##95;	_	127	7F	177	##127;	DEL

Source: www.LookupTables.com

Here we can see that the number 4 represents the represents "EOT" character which we see displayed as `EEEE` Note the same data is always being sent, we are simply changing how it is displayed on the screen – as a character "EOT", or as the numerical binary data 4.

You can also have MATLAB show the actual ones and zeros corresponding to the number 4:

```
d1_binary =

10x3 char array

'100'
'100'
'100'
'100'
'100'
'100'
```

This is explained in a more detail in the section "Reading the Data with RealTerm"

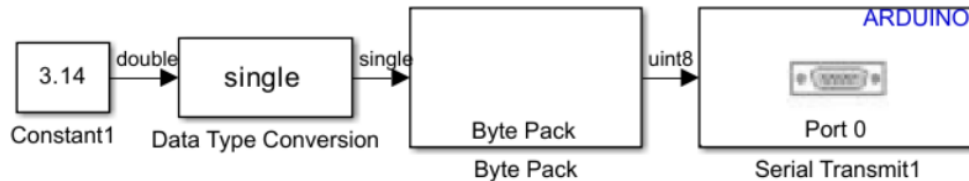
Part 2: Sending and Receiving Floating Point Data

Objectives:

- Use Simulink to send floating point data. Data can be sent and received faster than “Monitor and Tune” (external mode).

Simulink Model:

- Build the following Simulink diagram and deploy it to hardware: (note if your Simulink diagram does not show the block names by default you can right click the block and select ‘format’, ‘show block name’, ‘on’.)



This code will take the number 3.14, convert it to ‘single’ data type which used 4 bytes to represent a floating-point number. The block ‘Byte Pack’ separates it to single bytes that are sent to the serial port to transmit the data. After the code is downloaded run the following file in MATLAB to verify you can read the number 3.14.

```
%% Read from Serial port:
clc, clear all % clear screen, clear all variables
s = serialport('COM45',9600); % Specify your com port
% Open Serial Port:
fopen(s);
% Read 30 data points
d1=(fread(s, 10, 'single'))' % display data

clear s % close the COM port
```

This code order each of 4 bytes a floating-point single:

```
Command Window

d1 =

    3.1400    3.1400    3.1400    3.1400    3.1400    3.1400    3.1400    3
```

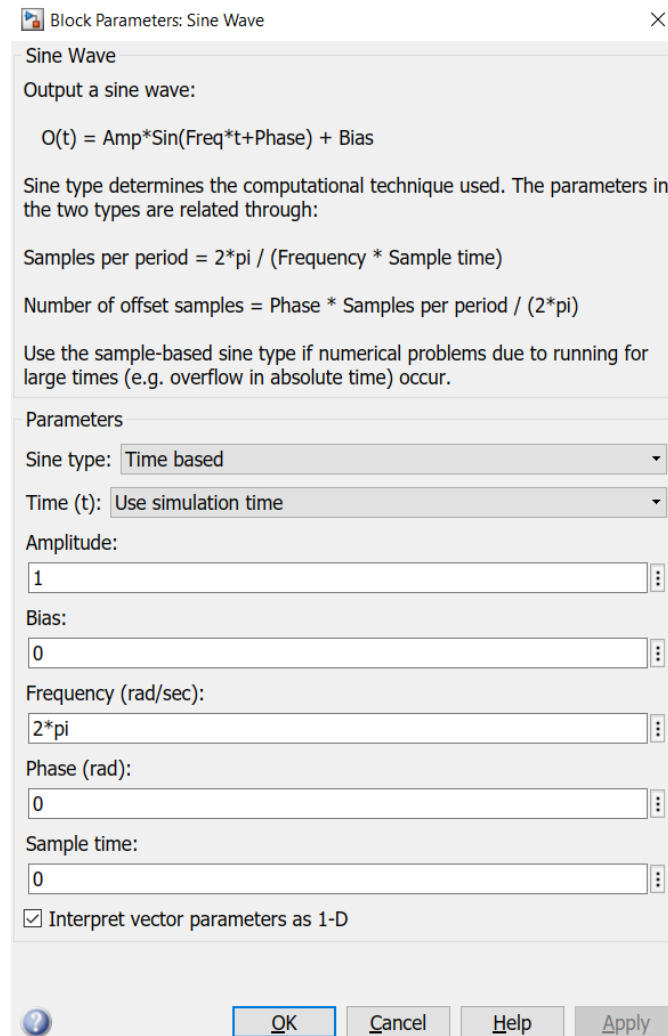
BAUD Rate Calculation

The default setting for the BAUD rate is 9600. If, for example, the sample time is 0.002 second, and we are sending a single which is 4 bytes. The number of bits (8 bites in a byte) per second is then $4 \times 8 / 0.002 = 16000$ bits per second. This means the minimum BAUD rate for a single data channel at 0.002 seconds is 1600. The default 9600 is more than fast enough.

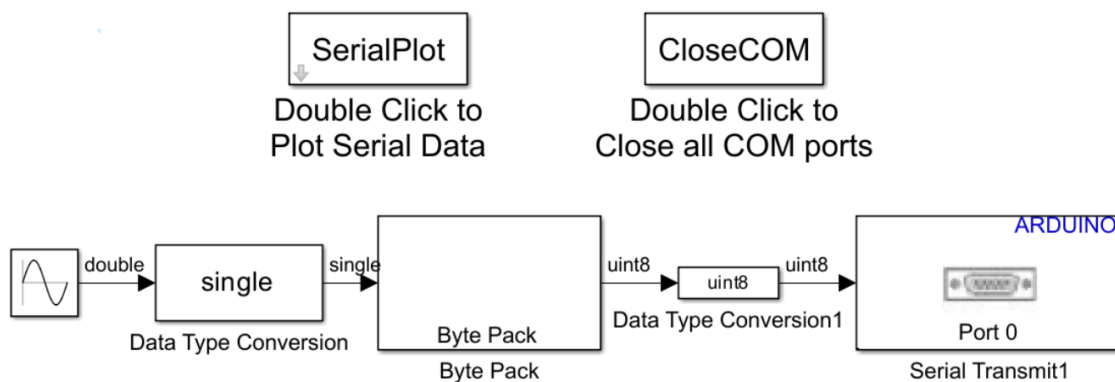
Plotting and Storing data

The previous m-files can be used as starting point to read, plot and store data.

Open the example “Sine_Wave_Single_Channel_Example.slx”. Make sure you also have the file “SerialPlot_25a_Timer.m” in your working directory. Change the sine wave frequency to 2π :



Next download the code to the hardware:



Note that the “Build, Deploy & Start” button downloads the code to the hardware only. After it downloads it will only execute the code you specify, in this case sending data from a sine wave to port 0 as a “single” floating point number (4 bytes).

- To capture and plot the data double click the “SerialPlot” block and set the following parameters. Be sure to use your COM port and the BAUD rate as specified in the configuration parameters.

Block Parameters: Double Click to Plot Serial Data

SerialPlot

Plot single channel data received from the serial line. View Matlab command window for connection and plot information. After stopping graph, the data is stored in the variables fdat and wdat in the workspace. Both variables are also saved to the file fdat.mat in the current directory. This block does not download any code to the hardware - just a block to plot data recieved. The plotting procedure can be modified by editing the file SerialPlot.m.

Parameters

COM Port:
14

Baud Rate (needs to be same as Baud Rate in configuration parameters)
115200

Number of samples to plot
500

text here

Data Type of data being sent

☐ uint8

☐ int16

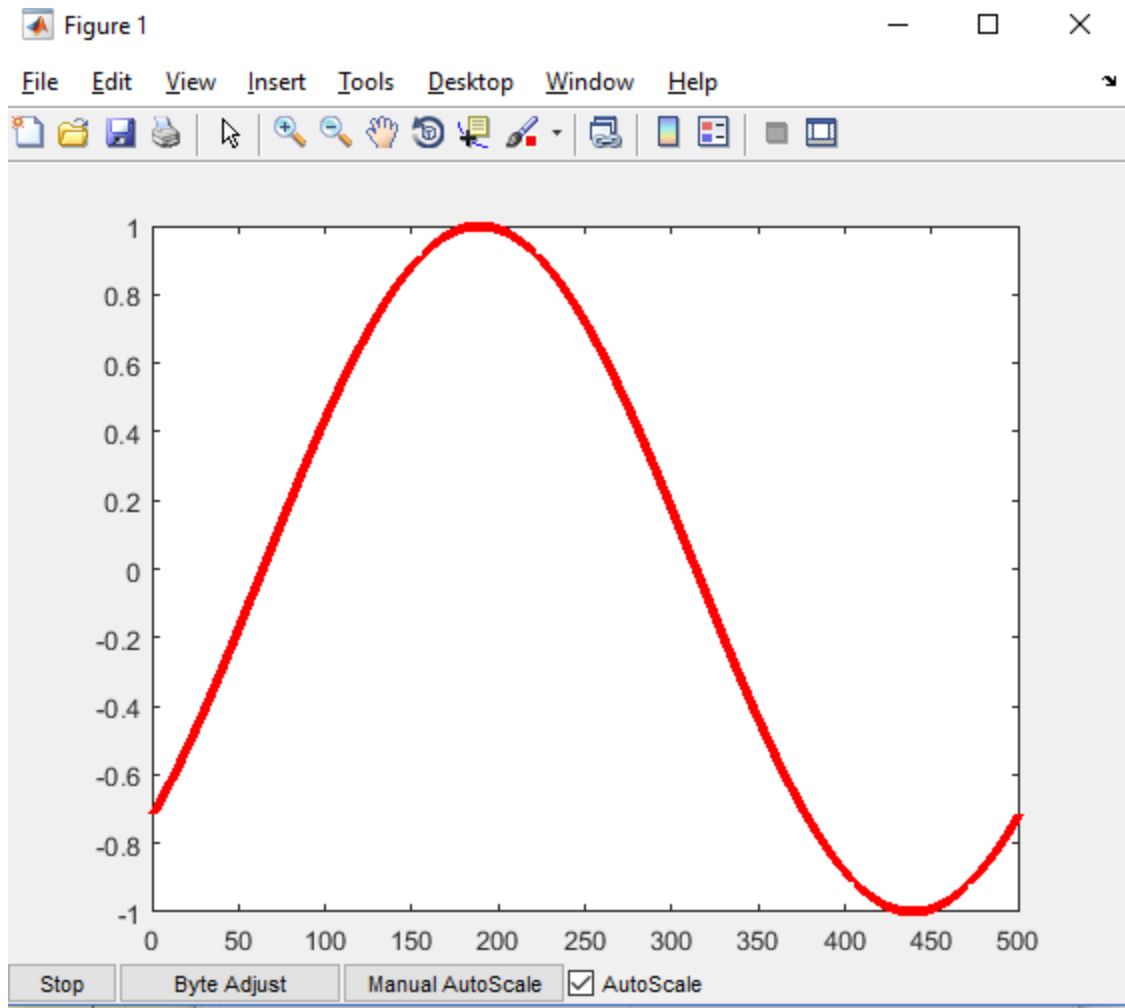
☒ single

Click Here to Plot Data

OK Cancel Help Apply

- Then click “Click Here to Plot Data”

You should observe a sine wave of frequency 2π rad/sec – which should repeat every second:



- Click “Stop” to stop recording data.
- The data is stored in file “dat.mat” in the current directory
- The complete data is stored in the fdat variable and the last window data in the wdata variable. You can then easily plot the stored data:

```
>> plot(fdat)
>> plot(wdat)
```

There are 3 possible data types you might want to send

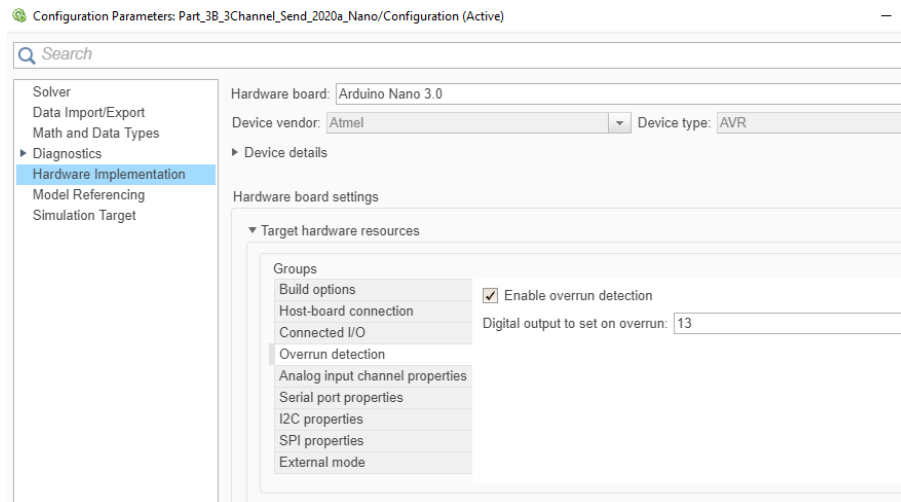
Type	Bytes	Range
uint8	1	0 to 255
int16	2	-32768 to 32767
single	4	-1.401298E-45 to 1.401298E-45

In general single should be used, but if data is needed faster other types can be used as long as the data is scaled to within the range of the data type.

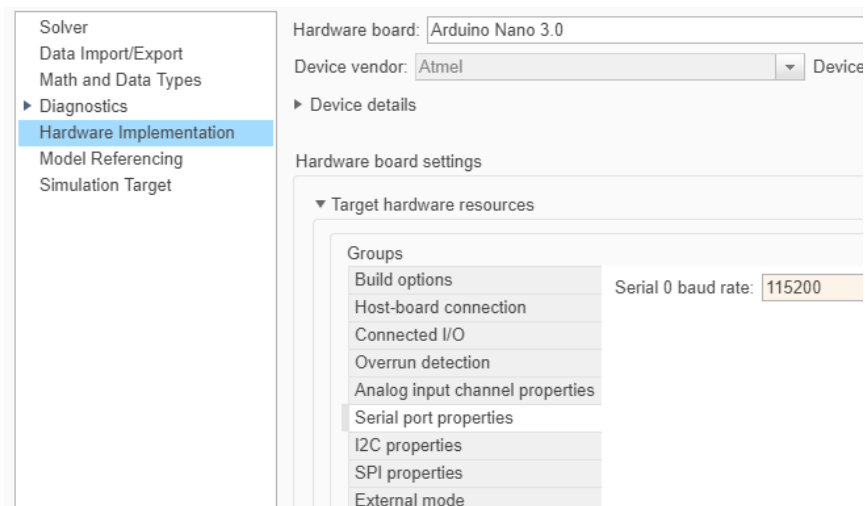
Sending Multiple Data channels

Open the example “Three_Channel_Example.slx” located in your Blackboard. Deploy to hardware and verify it plots all 3 channels.

Make sure overrun detection is on:



Baud rate is set to 115200



And the sample time is set to 5ms:

Figure 1 shows the MATLAB/Simulink environment for the 'Three_Channel_Example' model. The left pane displays the Simulink block diagram, which includes a scope 'SerialPlot' and a 'CloseCOM' block. The right pane shows the MATLAB script for 'Three_Channel_Example.m', which includes code for reading from a serial port, converting data to single precision, and sending it to a serial transmit block. The bottom pane shows the MATLAB Command Window with the execution results.

The MATLAB script 'Three_Channel_Example.m' contains the following code:

```

1 % Read from Serial port:
2 % Clear all: % Clear screen, clear all variables
3 s = serialport('COM4', 9600); % Specify your com port
4 % Open Serial Port:
5 fopen(s);
6 % Read 10 data points
7 display(sprintf('Data: %s', single('))) % display data
8
9 clear % % Close the COM port

```

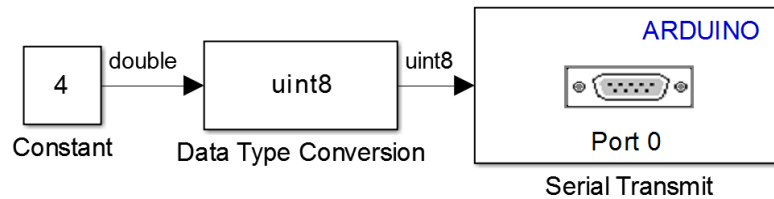
The Command Window shows the following output:

```

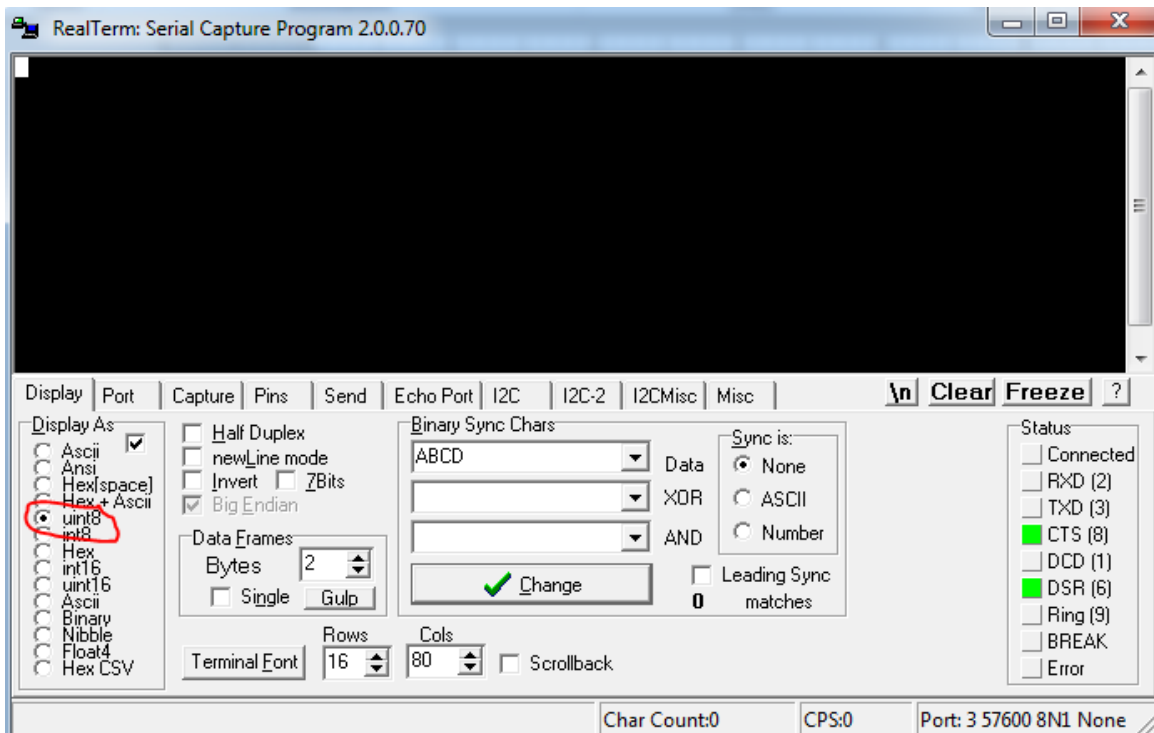
Data: [10 11 12 13 14 15 16 17 18 19]
Data: [10 11 12 13 14 15 16 17 18 19]
Data: [10 11 12 13 14 15 16 17 18 19]
Data: [10 11 12 13 14 15 16 17 18 19]

```

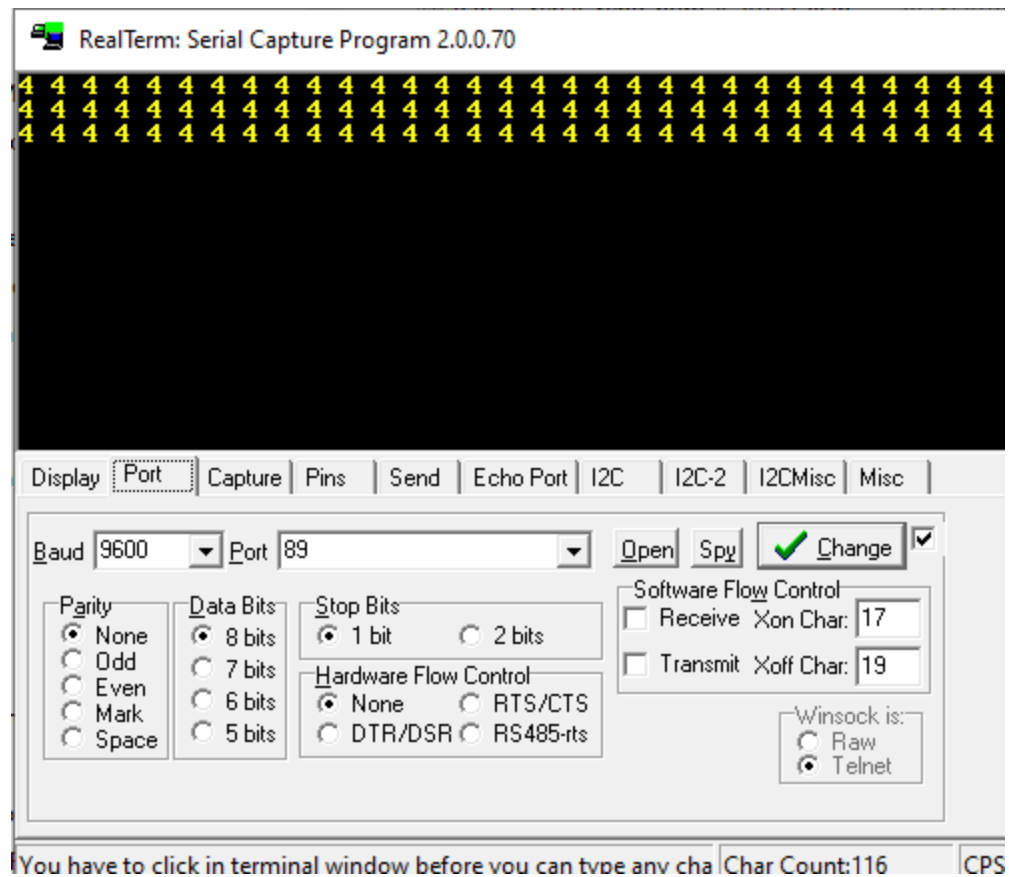
Another way to obtain data is RealTerm – although any serial terminal program will work. First download the following code to the Arduino:



After installing RealTerm open it in administrator mode.



- On the left select “uint8”. This indicates to RealTerm that the data type to expect is uint8 and it will display the results appropriately.
- Click the “port” tab, change the baud rate to 9600
- select the serial port of your hardware
- If the Open tab is depressed (as in the figure below) click it once to “close” the port, and again to “open” it. It should then start displaying data:



By specifying “uint8” RealTerm knows the data type so it can display it correctly.

Change the “Display As” to “Ascii” and open the port again:

You notice it will display the characters that the bytes represent instead of the numerical binary values. The characters represented by a byte can be found from a Ascii character table:

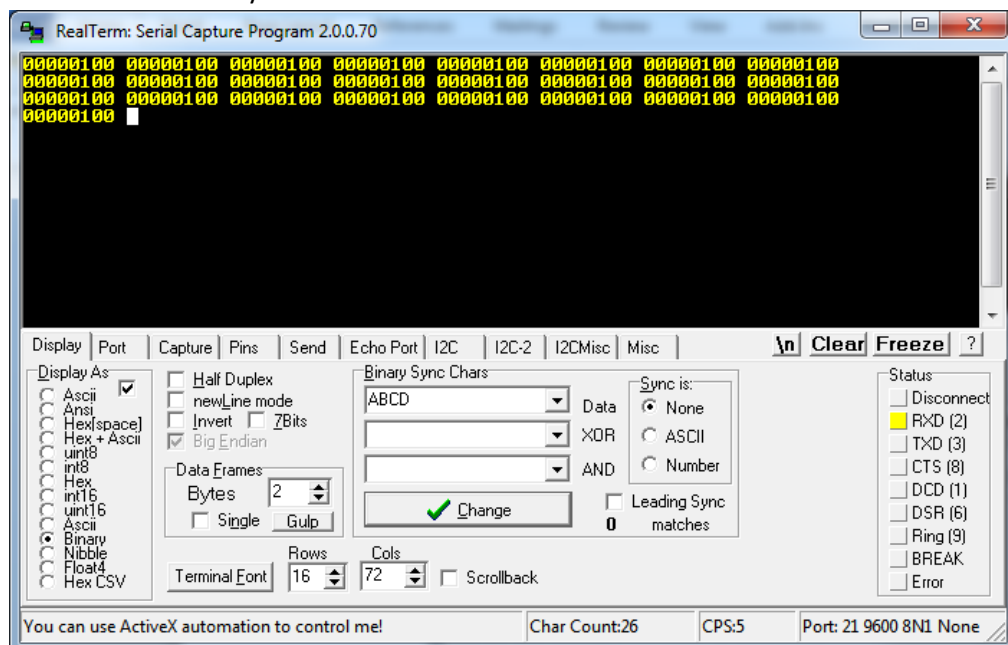
Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	##32; Space		64	40	100	##64; @		96	60	140	##96; `	
1	1	001	SOH (start of heading)	33	21	041	##33; !		65	41	101	##65; A		97	61	141	##97; a	
2	2	002	STX (start of text)	34	22	042	##34; "		66	42	102	##66; B		98	62	142	##98; b	
3	3	003	ETX (end of text)	35	23	043	##35; #		67	43	103	##67; C		99	63	143	##99; c	
4	4	004	EOT (end of transmission)	36	24	044	##36; \$		68	44	104	##68; D		100	64	144	##100; d	
5	5	005	ENO (enquiry)	37	25	045	##37; %		69	45	105	##69; E		101	65	145	##101; e	
6	6	006	ACK (acknowledge)	38	26	046	##38; &		70	46	106	##70; F		102	66	146	##102; f	
7	7	007	BEL (bell)	39	27	047	##39; '		71	47	107	##71; G		103	67	147	##103; g	
8	8	010	BS (backspace)	40	28	050	##40; (		72	48	110	##72; H		104	68	150	##104; h	
9	9	011	TAB (horizontal tab)	41	29	051	##41;)		73	49	111	##73; I		105	69	151	##105; i	
10	A	012	LF (NL line feed, new line)	42	2A	052	##42; *		74	4A	112	##74; J		106	6A	152	##106; j	
11	B	013	VT (vertical tab)	43	2B	053	##43; +		75	4B	113	##75; K		107	6B	153	##107; k	
12	C	014	FF (NP form feed, new page)	44	2C	054	##44; ,		76	4C	114	##76; L		108	6C	154	##108; l	
13	D	015	CR (carriage return)	45	2D	055	##45; -		77	4D	115	##77; M		109	6D	155	##109; m	
14	E	016	SO (shift out)	46	2E	056	##46; .		78	4E	116	##78; N		110	6E	156	##110; n	
15	F	017	SI (shift in)	47	2F	057	##47; /		79	4F	117	##79; O		111	6F	157	##111; o	
16	10	020	DLE (data link escape)	48	30	060	##48; 0		80	50	120	##80; P		112	70	160	##112; p	
17	11	021	DC1 (device control 1)	49	31	061	##49; 1		81	51	121	##81; Q		113	71	161	##113; q	
18	12	022	DC2 (device control 2)	50	32	062	##50; 2		82	52	122	##82; R		114	72	162	##114; r	
19	13	023	DC3 (device control 3)	51	33	063	##51; 3		83	53	123	##83; S		115	73	163	##115; s	
20	14	024	DC4 (device control 4)	52	34	064	##52; 4		84	54	124	##84; T		116	74	164	##116; t	
21	15	025	NAK (negative acknowledge)	53	35	065	##53; 5		85	55	125	##85; U		117	75	165	##117; u	
22	16	026	SYN (synchronous idle)	54	36	066	##54; 6		86	56	126	##86; V		118	76	166	##118; v	
23	17	027	ETB (end of trans. block)	55	37	067	##55; 7		87	57	127	##87; W		119	77	167	##119; w	
24	18	030	CAN (cancel)	56	38	070	##56; 8		88	58	130	##88; X		120	78	170	##120; x	
25	19	031	EM (end of medium)	57	39	071	##57; 9		89	59	131	##89; Y		121	79	171	##121; y	
26	1A	032	SUB (substitute)	58	3A	072	##58; :		90	5A	132	##90; Z		122	7A	172	##122; z	
27	1B	033	ESC (escape)	59	3B	073	##59; ;		91	5B	133	##91; [		123	7B	173	##123; {	
28	1C	034	FS (file separator)	60	3C	074	##60; <		92	5C	134	##92; \		124	7C	174	##124;	
29	1D	035	GS (group separator)	61	3D	075	##61; =		93	5D	135	##93;]		125	7D	175	##125; }	
30	1E	036	RS (record separator)	62	3E	076	##62; >		94	5E	136	##94; ^		126	7E	176	##126; ~	
31	1F	037	US (unit separator)	63	3F	077	##63; ?		95	5F	137	##95; _		127	7F	177	##127; DEL	

Source: www.LookupTables.com

Here we can see that the number 4 represents the represents “EOT” character which we see displayed. Note the same data is always being sent, we are simply changing how it is displayed on the screen – as a character “EOT”, or as the numerical binary data 4 – but they ones and zeros are always the same.

To see the actual ones and zeros

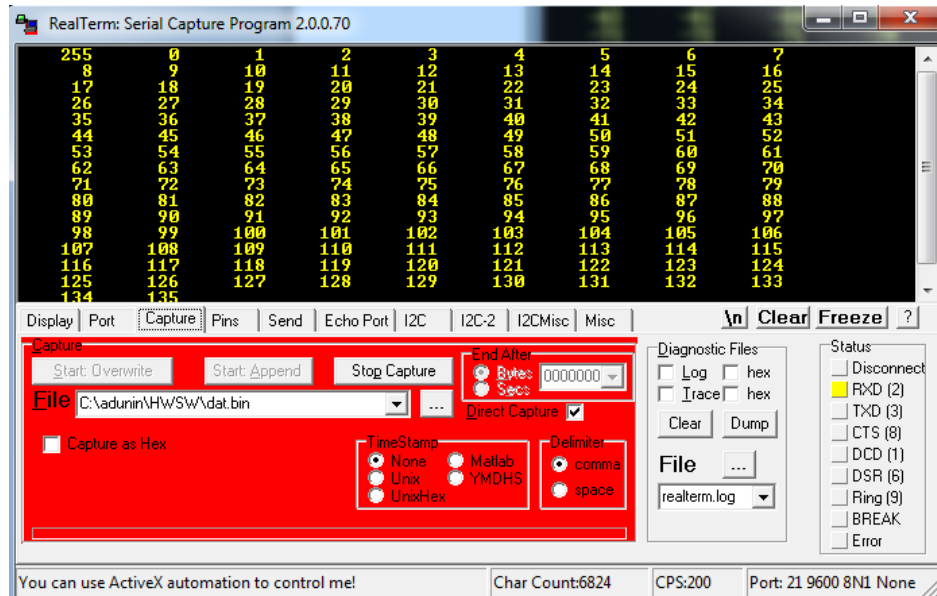
- go back to the Display tab and select Binary



Now the actual binary representation of the data is seen: 00000100 is the number 4, or the character “EOT”, but the ones and zeros are always the same.

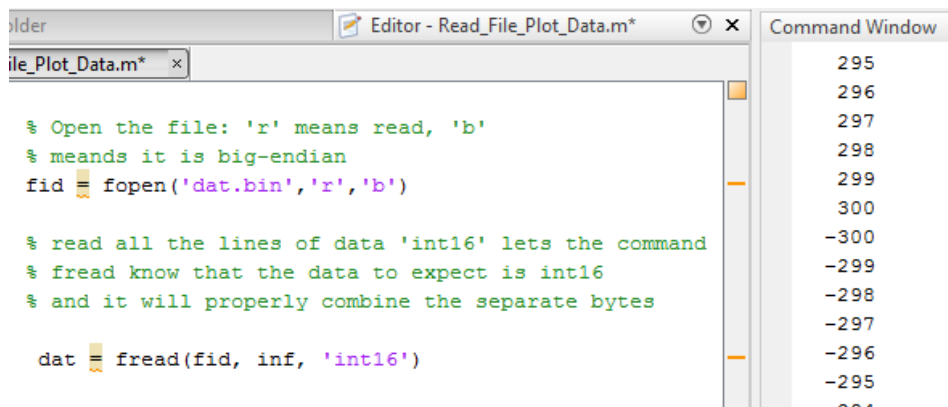
Writing the data to a file with RealTerm:

- Open RealTerm, set the baud rate and port, and connect
- Click the “Capture” tab
 - Enter the location and filename you want to store the data (your current Matlab directory). Since the data is in binary the filename should end in .bin to reflect this
 - Click the “Start Overwrite” button to begin writing data to the file



Reading the data file with Matlab:

Write the following M-file to open the file and read the bytes in the appropriate format:



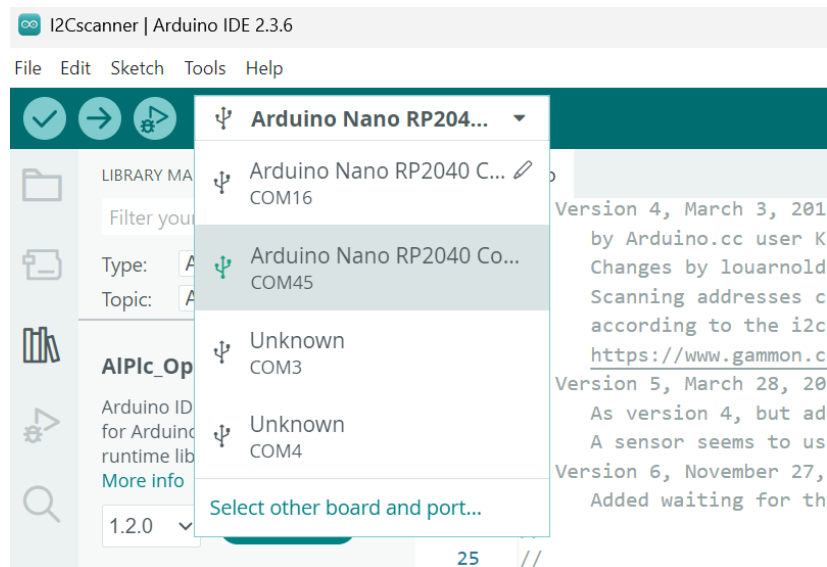
Part 4: Sending data with the Arduino IDE

Objectives:

- Use the Arduino IDE to send ascii encoded data and binary data

Arduino Code:

- Open the Arduino IDE (this should have been installed when the Simulink Arduino Support Package was installed)
- Select your target board:



- Open a basic serial example:

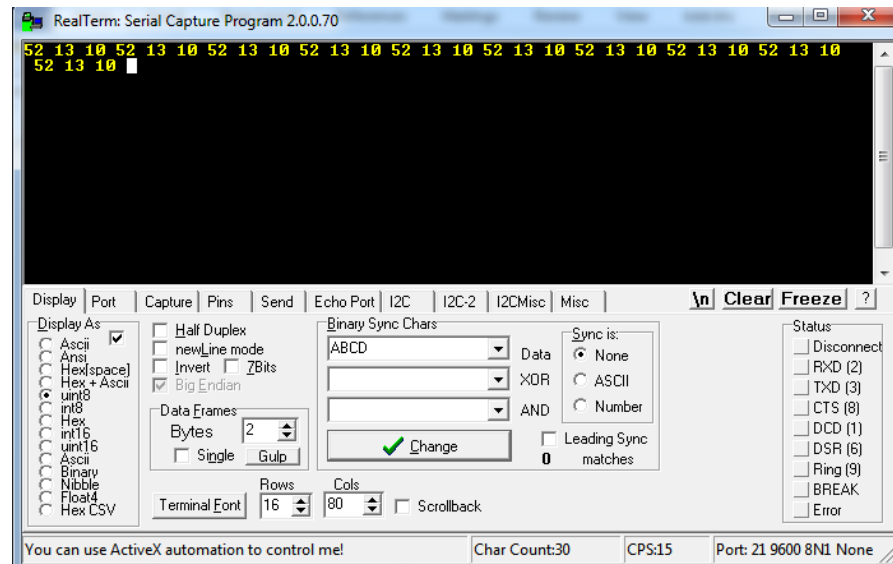


- **Modify** the code to send only the number 4:
 - Download the code to the board with the Right arrow
 - Note – you must make sure the serial port from RealTerm is closed – it cannot download code when the serial port is open in another application

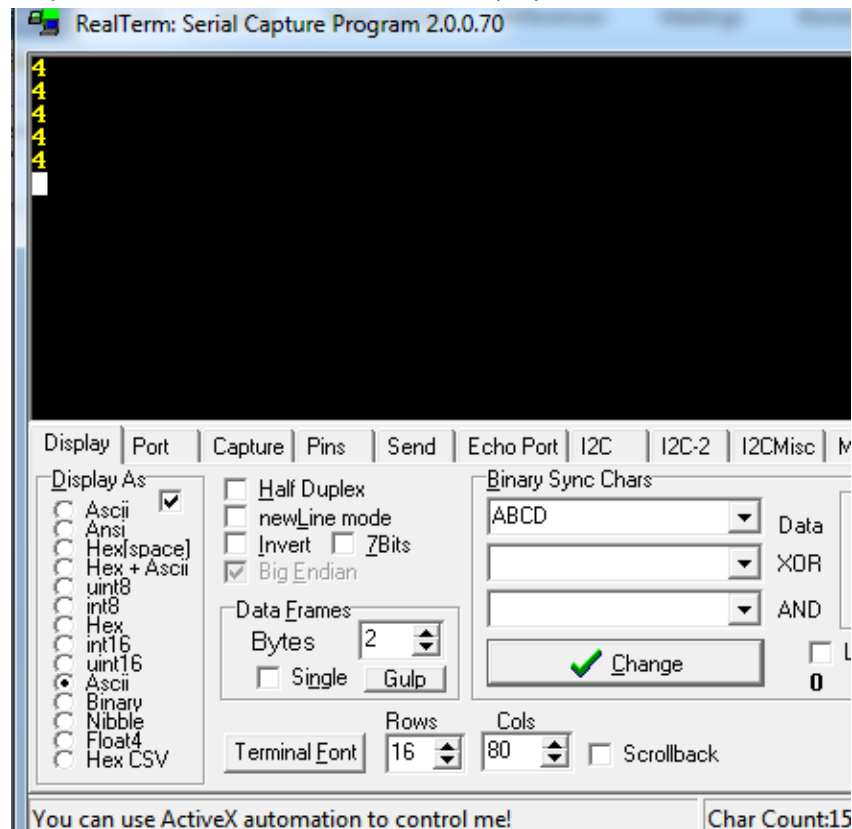
```
// the setup routine runs once when you press reset:
void setup() {
  // initialize serial communication at 9600 bits per second:
  Serial.begin(9600);
}

// the loop routine runs over and over again forever:
void loop() {
  // read the input on analog pin 0:
  int sensorValue = 4;
  // print out the value you read:
  Serial.println(sensorValue);
  delay(1); // delay in between reads for stability
}
```

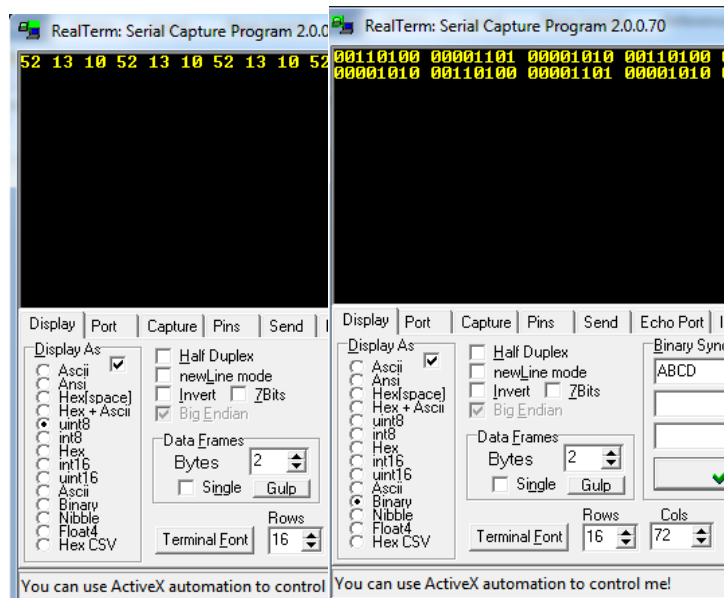
- Open the terminal program to view



The data is not as expected. Now click on Ascii for the Display as format



The display shows 4. When the 'Serial.println()' command is used the data is encoded to human readable ASCII characters. The data (4) is encoded to the character '4' which is represented in decimal as 52 (101010). In addition each 4 is written on a new line which is a carriage return followed by a new line 13 (00001101) and 10 (00001010):



If you want to write the data in binary (that is 4 as 0000100) use the Serial.Write command.

Questions

An analog sensor records a value 16 (the result of the Analog to Digital conversion). You want to send this data (the number 16) from the microprocessor to your computer.

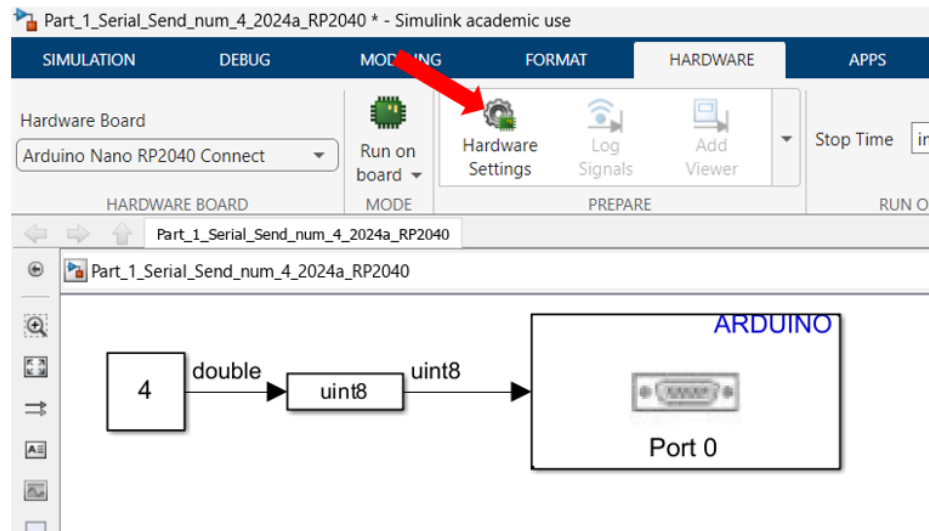
- How many bytes does it take to send this data in binary format?
 - What is the binary (ones and zeros) representation of this data?
- How many bytes does it take to send this data ASCII encoded?
 - What is the binary (ones and zeros) representation of the ASCII encoded data?

Now the sensor reads 32768.

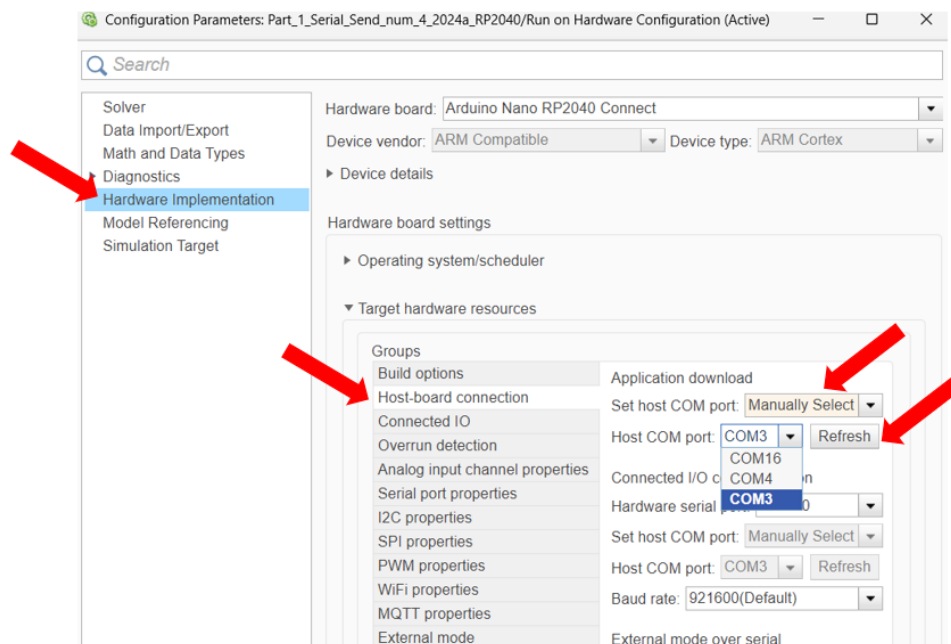
- How many bytes does it take to send this data in binary format?
 - What is the binary (ones and zeros) representation of this data?
- How many bytes does it take to send this data ASCII encoded?
 - What is the binary (ones and zeros) representation of the ASCII encoded data?
- Which of these formats would be “fastest” to send?

How do I find my COM port?:

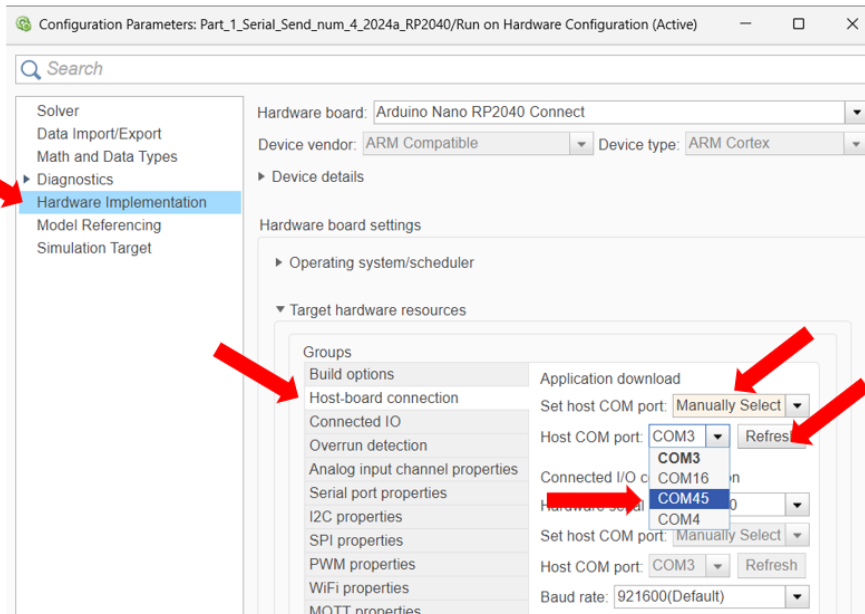
First disconnect your device. Make sure your device is NOT connected. In Simulink open Hardware Settings:



Navigate to 'Hardware Implementation', 'Host-board connection', 'Set host COM port', 'Manually Select', 'Refresh'. Then expand the drop down box and take note of all the com port numbers. In the example below the numbers are 3, 4 and 16.



Now plug in your device. Make sure your device IS connected. Refresh the 'Host COM port' a couple times and take note of the new number that should appear.



The new device that appears is 'COM45'.